



Slide 1

CS 170

Java Programming 1

Midterm Exam 1

Duration: 00:01:22
Advance mode: Auto

Notes:

Hi Folks,

This is the CS 170, Java Programming 1 lecture on Midterm Exam 1.

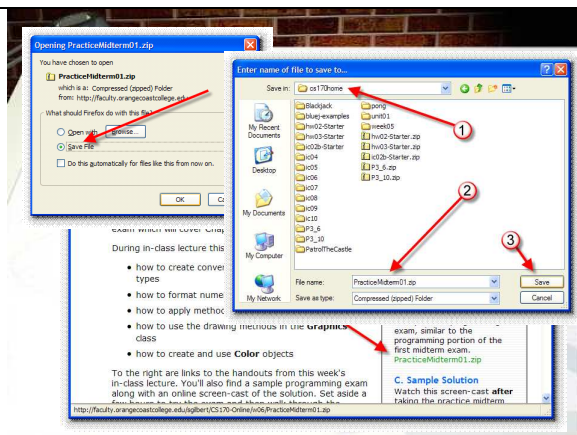
The CS 170 exams consist of two parts, a 50-question, closed-notes, closed book short-answer, multiple-choice exam, and a practical programming exam. This lecture will show you how to download and set up a practice programming exam that is similar to the problem you'll need to complete for the first midterm.

Many students approach programming exams the same way that they approach a history or psychology exam—by trying to memorize large quantities of material, and then recall the material when the time comes. That strategy doesn't work very well for programming exams. Instead, you have to think through the problem, and then build up your solution piece-by-piece.

Since you have about an hour and a half to complete the programming portion of the exam, you'll want to follow the instructions in this lecture to get started, and then time yourself to see how you've done. Practicing under test conditions will really help you when it comes to the day of the test.

Once you've finished taking the practice exam, come back and watch the second-half of this lecture to double-check your work.

Ready? Let's get started.



Slide 2

Download

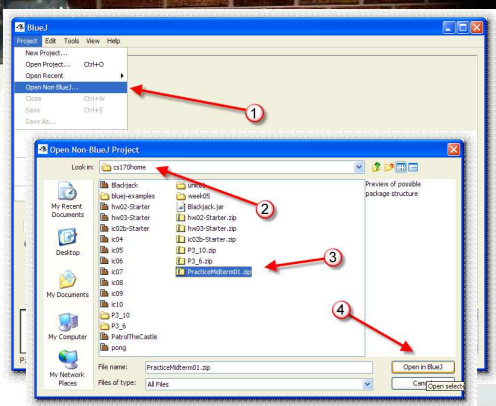
Duration: 00:00:51
Advance mode: Auto

Notes:

Get started by downloading and opening the sample exam. The practice exam is stored as a zip file, but you don't need any "unzipping" software to open in. Instead, you'll use BlueJ.

To get the starter file, just click the link named PracticeMidterm01.zip shown here.

Rather than opening the file, choose Save as shown here. Depending on how your browser is set up, you may, or may not be given an option as to where to save the file. If you're given a choice, then save it in your cs170home folder, like this. If, on the other hand, your browser saves the file on your desktop automatically, then you'll need to drag it into your cs170 folder before you go to the next step.



Slide 3 Setup

Duration: 00:00:59

Advance mode: Auto

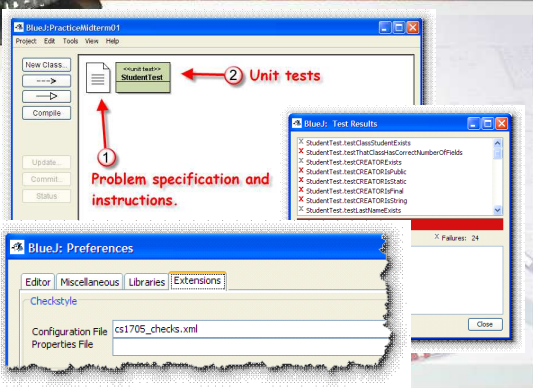
Notes:

To open the exam, start BlueJ and then select Open Non-BlueJ as shown in the slide here.

When the "Open Non BlueJ Project" file-chooser dialog appears as shown here:

- navigate to the cs170home folder, where you downloaded or copied the PracticeMidterm01.zip file.
- select the PracticeMidterm01.zip file
- and finally, choose Open in BlueJ.

When you do this, BlueJ will automatically expand or unzip the PracticeMidterm01.zip file and create a BlueJ project (that is, a folder) with the same name. That means, if you need to do this step twice, you'll need to rename the BlueJ project before BlueJ can expand the zip file once again.



Slide 4 Unit Tests

Duration: 00:01:17

Advance mode: Auto

Notes:

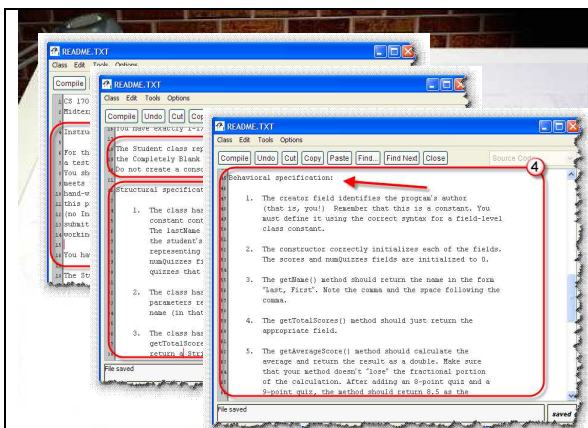
When BlueJ first opens the exam, you'll see the familiar project window with two icons. The icon that looks like a sheet of note-paper with a folded corner contains the problem specification along with the exam instructions. That's marked (1) in the slide shown here. You can open it just by double-clicking.

The greenish icon with the heading "unit test" contains a copy of the file that's used to test your program's correctness. Don't bother opening this file. Instead, just click the Run Tests button to see its effect.

As you can see, since you haven't written any code yet, all of the tests fail.

In addition to the unit tests, you'll need to use the Checkstyle plug-in to make sure that your code meets the CS 170 style guidelines. About 10% of your grade will be based on these style guidelines. Since you'll be taking the exam using a special login-ID, you might need to configure Checkstyle to use the correct file, cs1705_checks.xml, like this.

Go ahead and open the specification file and we'll take a look at that next.



Slide 5

Specification

Duration: 00:02:13
Advance mode: Auto

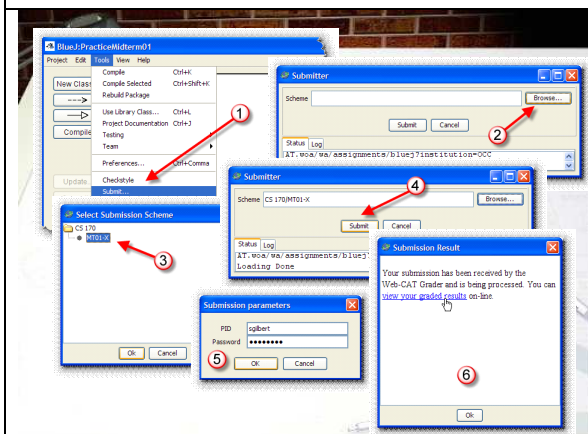
Notes:

The first part of the README.TXT file simply contains the exam instructions. This is an open-notes, open-book exam. Notes must be hand-written, and you can't get help from a friend (or from strangers) or use programs that you've written previously. (Programs copied into your notes are fine.) You can also use the online JavaDocs. You'll have an hour and a half to complete the programming exam (more if you complete the written portion early, though).

Below the instructions is a short summary (marked 2 in the slide) that tells you the name of the class. In this case, you're asked to create a class named Student and to use the Completely Blank File BlueJ template. Make sure you pay special attention to the name of the class. If you name your class something other than what I've requested, it won't pass any of the tests. I also don't really care if you use one of the other BlueJ templates (like the Bare Standard Class), but I find it easier just to start from scratch.

Below the summary is the structural specification. Here you'll find the names and types of each of the fields, the number and types of the parameters passed to the constructor, and the names, return types and parameters required for each method. Each method, constructor and field must exactly match this structural specification and each item must be syntactically correct before your program can be graded.

Finally, at the very bottom is the behavioral specification. In this section, you'll find out what each method is supposed to do. I usually test each method in several different ways, so even if you can't exactly match the desired behavior, you may get partial credit by implementing some of it. That's not true for the structural specification, though. If I ask you to create a method named doSomething() that returns an int and takes a String parameter, you won't get any credit at all if your method is spelled incorrectly, or if the return type or parameter type are incorrect.



Slide 6

Submission

Notes:

Once you've passed all of the tests (or the time is called on the exam) and you've checked your program with Checkstyle, you'll turn it in using Web-Cat and the BlueJ submitter.

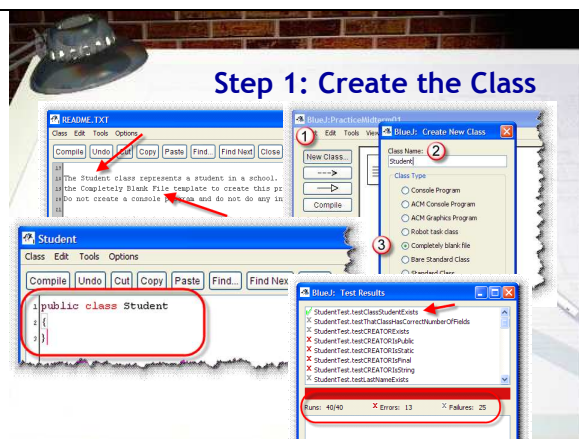
- On the Tools menu, simply choose Submit
- When the Submitter dialog appears, click Browse
- On the Select Submitter Scheme dialog, expand the CS 170 folder and locate the correct exam. For the sample exam, it's named MT01-X. It will be different for the actual exam, of course.
- Click the Submit button once the Scheme text-box has been filled in.

Duration: 00:01:34
Advance mode: Auto

- Enter your Web-Cat ID and password (the same as for your homework) and click OK.

If all went well, your submission will be uploaded and you can click the hyperlink shown here to view your submission in your Web browser. If you have Web-Cat already open, you don't need to click the hyperlink. Just click the OK button to dismiss the dialog instead.

Now, before I go ahead and walk through the solution, why don't you close this presentation and try taking the exam yourself. Minimize distractions and limit yourself to an hour and a half. When you're done, check your work against the solution shown in the second half of this presentation.



Slide 7
Step 1: Create the Class
Duration: 00:01:16
Advance mode: Auto

Notes:

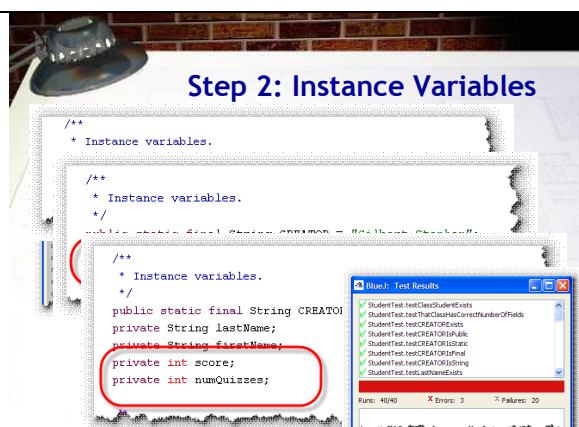
All finished? OK, now let's walk through the solution.

The summary asks you to create a class named Student using the Completely Blank File template. To solve this part, you need to do two things:

- First, create the file using the New Class button, filling in the class name (Student) and choosing Completely Blank File from the list of Class Types before clicking OK.
- Then, open the file and type in the basic structure for a class. You should have this memorized and be able to type this in by "rote". Since the class is named Student, you'll type in public class Student and add the braces that delimit the body.

Compile your file and run the tests, and you'll see that two of your tests now pass (although one is a "false positive").

You might also want to add your comments as you go, so you don't have a big job right at the end. Remember that you'll need a file comment that includes an @author and an @version tag. You can see my comment on the subsequent slides.



Slide 8

Notes:

The first item in the structural specification is the instance variables or fields. 1. As you can see, the class has five fields. Let's look at them.

- The first one is a field named CREATOR that is a String constant. Since it's a constant, it needs to be initialized when it is created. You're instructed to initialize it with your name, in the form "Last,First", with no spaces. Remember that the syntax for a String constant that is an instance variable is public static final String CREATOR = "Gilbert,Stephen"; Because it is a constant, we use capitals to name the field.

Step 2: Instance Variables

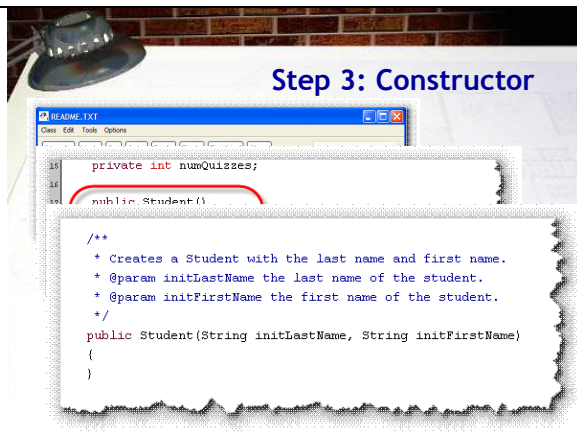
Duration: 00:02:08

Advance mode: Auto

- The next two fields are String, lastName and firstName, that represent the student's name. Here's the syntax you'll use for that. Remember we always make our fields private except for public constants. Remember also that you need to spell the field names exactly as the specification describes.
- The final two fields are int types. The field named scores will represent the student's total quiz scores while the field named numQuizzes represents the number of quizzes that the student has taken. Again, you must spell these exactly as specified, including all capitalization.

Run the tests at this point and you'll see you have only 23 errors out of 40. (Note though, that the number of errors don't correspond exactly to the number of points on the exam. Simple structure tests are worth fewer points than the more complex behavioral tests.)

If you scroll down through the tests, you'll see that I've misspelled the name of the field scores and so three of the tests that should have passed failed. Use the Run tests button frequently to find simple typos like this as soon as possible.



Slide 9

Step 3: Constructor

Duration: 00:01:50

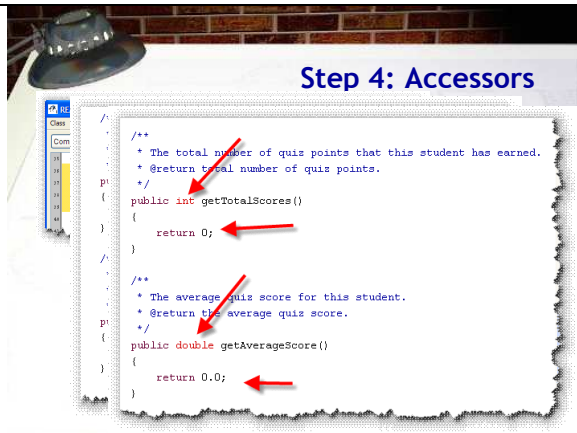
Advance mode: Auto

Notes:

Once you have the fields done, the next step is write the constructor for the class. At this point, you don't need to worry about what the constructor does (that is, how the fields are initialized). During the "structural" part, all you're worried about is getting the return type and parameter correct.

- The structure for a constructor is pretty simple. It must have the same name as the class and it must have NO return type. Many students mistakenly try to make the constructor a void method. Don't do that. Here's the basic structure for the constructor.
- Next, notice that the Student constructor takes two formal parameters. For each parameter you need to supply a type and a name, separated by commas. The two parameters we need are both Strings and they'll be used to pass in the last name and first name. To do this, your code should look something like this. To make things easier, you'll want to use a name that's similar to the field name that the parameter will initialize. I've used the prefix "init" which stands for initial, like the programs we saw in class. Remember, also, that the order of the parameters is significant, and must match the specification.
- Finally, to finish up, you'll want to add the JavaDoc comment. Remember you need an @param tag for each parameter. Here's an example.

Notice that we don't worry about the actual initialization at this point. We'll come back to that when we do the behavioral part.



Slide 10
Step 4: Accessors
 Duration: 00:01:51
 Advance mode: Auto

Notes:

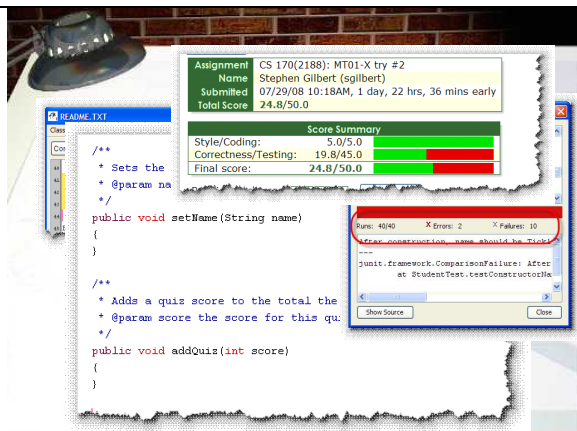
The next step is to take care of the accessor methods. Remember that accessors are methods that tell us something about an object, usually something about the state stored in its fields. At this point, though, all you care about is the syntax, and for that you have to remember two simple rules:

- every accessor has a return type that is something other than void
- every accessor ends with a return expression of the same type as the method return type.

Let's see how that works.

Our first two accessors, getName() and toString() are both String type methods. To create the skeleton for these two methods, you simply write private String (because that's the return type), followed by the method name, a set of parentheses and a set of braces for the body. In the body, just return the empty String. (Remember, we're not concerned about correct behavior, just the correct structure.) Here's what these two methods should look like. I've added the JavaDocs in the slides.

The structure for the getTotalScores() and getAverageScore() methods is identical, except that the return types are int and double respectively, and the return expressions for each method must match the return types. While working on the structure, I'll return 0 for getTotalScores() and 0.0 for getAverageScores(). Here's what these should look like.



Slide 11
Step 5: Mutators
 Duration: 00:01:26
 Advance mode: Auto

Notes:

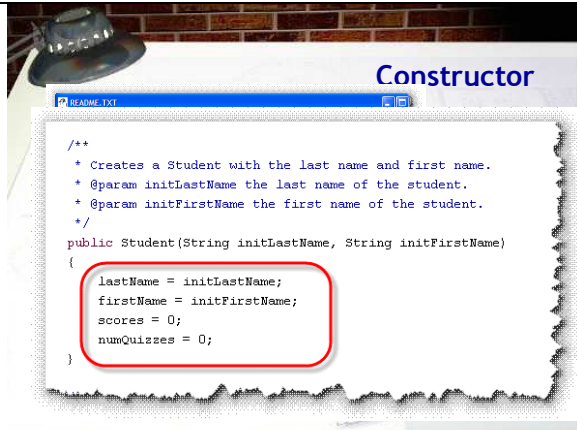
The last step in the "structural" portion of the exam is to write the two mutators described here. As you can see, both methods have no return value, which means you'll use void for the return type and won't use a return expression. Both methods, though, require you to define parameters. The addQuiz() method requires a parameter of type int and setName() requires a parameter of type String.

Here's what the basic skeleton for each method should look like. Note that I've added the @param tag for each parameter to the JavaDoc. I, personally, find it easier to write these as I go, instead of waiting until the end.

At this point, you should be able to click the Run Tests button and all of the "structural" tests should pass. In the sample program, only 12 of the tests still fail, and all of them have to do with the behavior of the class.

Even though that means we've completed 28 of 40 tests, though, when submitted to Web-Cat the score should be about 50%. Generally, getting the structure (that is, the syntax) correct will be worth about half of the points and getting the behavior correct will be worth the other half.

Let's go tackle that now.



Slide 12

Constructor

Duration: 00:01:24

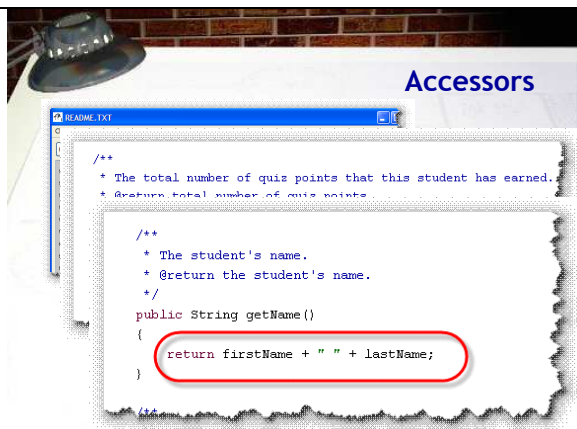
Advance mode: Auto

Notes:

Scroll on down to the behavioral specification and you'll see that we already completed step 1 when we defined the CREATOR field, so let's move on to number 2: correctly initializing the class in the constructor.

To initialize each of the fields, just use assignment to copy the parameters `initLastName` and `initFirstName` into the corresponding fields like this. Make sure when you do this that the parameter variable always appears on the right-hand side of the assignment operator, and the field name always appears on the left. If you reverse that order, you won't get a syntax error, but a silent logic error.

Even though we don't have parameters for `scores` and `numQuizzes`, the specification asks you to initialize them to 0. (Actually, they are already automatically initialized to 0 as instance variables, but it doesn't hurt anything to explicitly initialize them as I've done here. It's actually a good habit to get into, because many of you will move to C++ where instance variables are not implicitly initialized like they are in Java. If you get in the habit of explicitly initializing every field, you won't learn a bad habit that you'll have to unlearn later.)



Slide 13

Accessors

Duration: 00:00:52

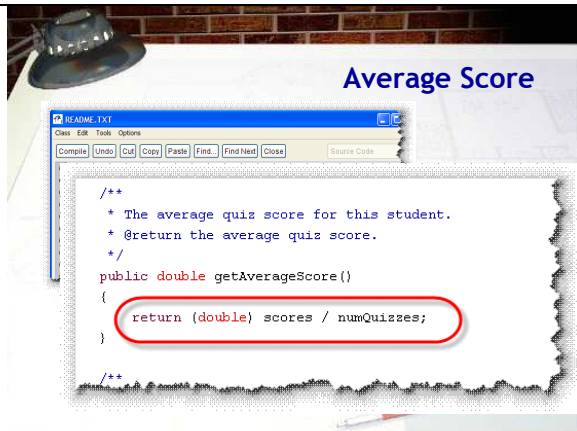
Advance mode: Auto

Notes:

Let's start with the first two accessors, `getName()` and `getTotalScores()`, since both of them are pretty simple.

The `getTotalScores()` method simply returns the appropriate field, in this case `scores`. Just replace the `return 0` with `return scores` like this.

The `getName()` method requires you to concatenate the two fields `firstName` and `lastName` together, separated by a space. To do that, just replace the `return ""` with `return firstName + " " + lastName`; as I've done here.



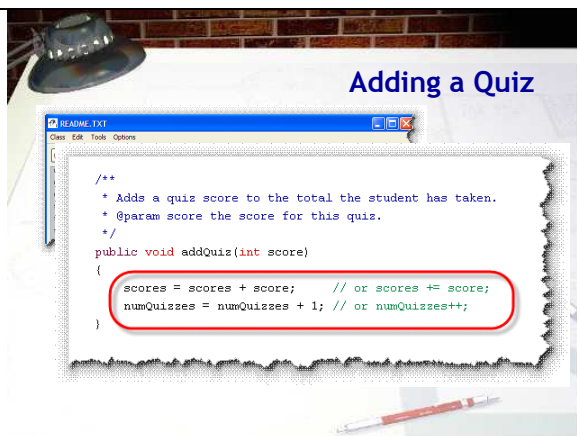
Slide 14
Average Score
 Duration: 00:00:59
 Advance mode: Auto

Notes:

The getAverageScore() accessor method is a little more complex because it needs to return a double, but the two instance variables used in the calculation, scores and numQuizzes are both ints. There are a couple of ways to solve this, but the easiest way is to cast either scores or numQuizzes to a double when you do the calculation, like this.

Note that you must make sure you don't put parentheses around the scores / numQuizzes calculation because then you'd have an integer division with the result cast to a double. We want to cast scores to a double so that numQuizzes is automatically promoted and the calculation is done as a floating-point, not integer division.

Note also that you can't yet test to see if this method works, because we haven't finished the addQuiz() mutator method.



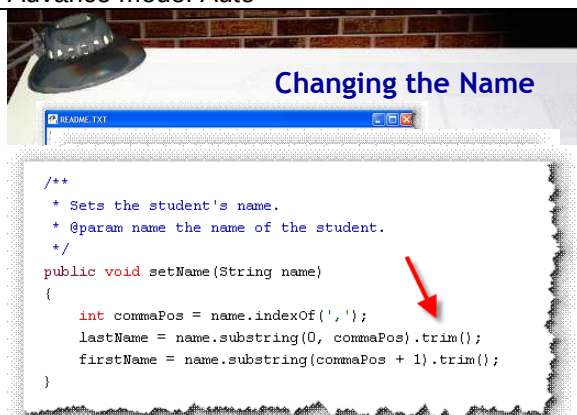
Slide 15
Adding a Quiz
 Duration: 00:00:56
 Advance mode: Auto

Notes:

Since we can't test getAverageScore() without adding a quiz, let's write the addQuiz() mutator method next. (It's also the easiest one. To do your best on the exam, do the easy stuff first, so you don't get stuck on some difficult problem, and then pass up some easy points.)

You'll remember that addQuiz() has one parameter, the int score, and the instructions say to add that parameter to the instance variable scores, and also, to update the number of quizzes taken. That means we just need to add the two lines of code shown here. Notice that you can use simple assignment or you can use the short-hand assignment and increment operators.

Once you make this change, the errors should go down to about 7 of 40.



Slide 16
Changing the Name
 Duration: 00:02:05
 Advance mode: Auto

Notes:

The last two methods are the most difficult. Let's just take them in order.

The setName() method is going to receive a new name for the Student in the form "Last,First". It's up to you to break it up into two pieces and put each piece into the appropriate field. You know that the name parameter will always contain a comma, but it may contain extra spaces either before or after the comma, and you'll have to remove them.

To solve this we need to:

- find the position of the comma
- put the portion of the string in front of the comma into lastName, and the portion after the comma into firstName
- remove any extra spaces from each name

To find the position of the comma, you first use the indexOf()

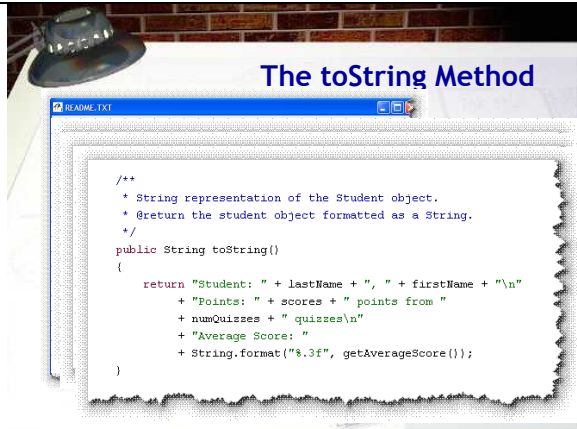
method like this.

Use the two-parameter form of substring to extract the portion of the string before the comma. Your substring will start at 0 and end at the comma position. Remember with substring, the ending position is not included as part of the string. Store this in the field `lastName` like this.

Now, use the one-parameter form of substring, starting at the character immediately following the comma, to initialize `firstName`, like this.

Finally, we need to remove any extra spaces at the end or beginning of the substring. You can do that using the `trim()` method. Rather than trimming the fields after you've assigned the substring, you can "chain" the `trim` call onto the end of the substring call like this.

Once you've done that, run the tests again and you'll see that everything passes except those related to the `toString()` method, which we have yet to complete.



Slide 17

The toString Method

Duration: 00:02:13

Advance mode: Auto

Notes:

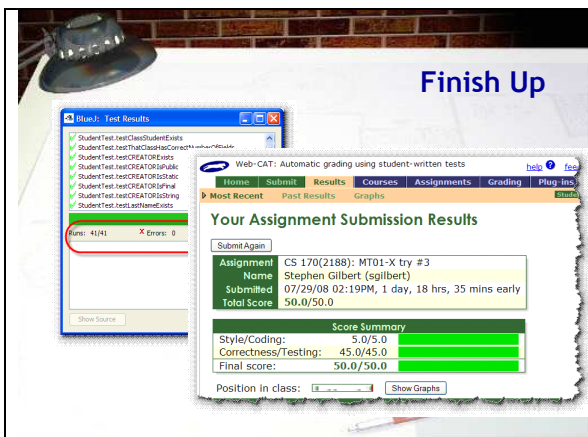
The `toString()` method returns a `String` representation of all of the data associated with a single student. The data needs to appear on three lines, but your method only returns a single `String`. That means you'll need to embed the newline escape character inside the `String` that you return.

The returned `String` also has a combination of fixed verbiage (such as the label `Student` that begins the first line) and variables, such as the number of points, the student name and the average score. The average score variable needs to be formatted so that three decimal places are displayed.

To start with, just use concatenation to paste together the information from the specification, along with the newline characters, like this. I just used copy and paste from the `README.TXT` file, then added in the quotes, the newlines and the concatenation characters. When you've done this, you'll see that only three tests fail.

Next, replace each of the underscore characters with a space and each of the data elements with field names, along with more concatenation. Here's what the code should look like after doing that. Notice that I've broken line 2 into two parts so that no lines are more than 80 characters long (so I don't lose any formatting points).

If you run the tests after this change, you'll see that only one test now fails, the one that has to do with the formatting of `getAverageScore()`. To format `getAverageScore()`, use the `String.format()` method with `"%.3f"` as the format specifier. You don't need a width, just the precision. Because the value you are formatting is a double, you use `"f"` for the type. Here's what the finished method should look like.



Slide 18

Finish Up

Duration: 00:00:25

Advance mode: Auto

Notes:

When you're done, run the tests one last time to see if you've forgotten anything. Use Checkstyle to check for style errors and then submit it to Web-Cat. If all goes well, you should get 50 out of 50 on this practice exam, and, I hope on the real exam as well.

See you at the midterm.